Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251198**
Nama Lengkap	**Moses Merohito**
Minggu ke / Materi	**09 / Pengolahan String dan Regex**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1 Pengantar String

String adalah rangkaian karakter yang digabung menjadi satu kesatuan dan digunakan dalam program komputer untuk menyimpan teks, baik yang pendek maupun panjang. String merupakan jenis data yang dapat menampung huruf atau karakter, dan biasanya disimpan dalam format kode ASCII. Tidak semua bahasa pemrograman menyediakan tipe data String secara langsung, contohnya bahasa C. String bukan termasuk tipe data dasar karena menyimpan lebih dari satu nilai dalam satu struktur. Di beberapa bahasa pemrograman, String dianggap sebagai kumpulan karakter, baik berupa array maupun daftar karakter.

MATERI 2 Pengaksesan String dan Pengolahan String

```
1   namasaya = "Antonius Rachmat C"
2   temansaya1 = "Yuan Lukito"
3   temansaya2 = 'Laurentius Kuncoro'
4   temansaya3 = "Matahari" + 'Bakti'
5
6   print(temansaya3)
7   print(namasaya[0]) #'A'
8   print(namasaya[9]) #'R'
9   print(temansaya1[1]) #'u'
10
11  huruf = temansaya2[0]
12  print(huruf) #'L'
```

String pertama kali dibuat dengan cara mendeklarasikan variabel sekaligus mengisinya dengan data. String bisa diakses secara keseluruhan cukup dengan menyebut nama variabel, atau bisa juga diakses per karakter dengan menggunakan indeks. Indeks pada string dimulai dari angka 0, mirip seperti pada struktur data list. Perlu diingat bahwa indeks yang digunakan harus berupa bilangan bulat, bukan desimal. Di dalam memori komputer, string disimpan secara berurutan dalam bentuk list yang berisi karakter-karakter dengan indeks yang dimulai dari nol.

b	a	n	a	n	a
[0]	[1]	[2]	[3]	[4]	[5]

MATERI 3 Operator dan Metode String

Operator in

Di dalam string itu dapat memeriksa apakah dalam sebuah kalimat merupakan substring dari sebuah kalimat lain menggunakan operator in. hasil dari pemeriksaan tersebut adalah True/False

```
1   kalimat = "saya mau makan"
2   data = "saya"
3   print(data in kalimat)  #True
4   print("mau" in kalimat) #True

5   print("dia" in kalimat) #False
```

Selain operator in pada string jua dapat melakukan comparison atau perbandingan yang juga menghasilkan True or False

```
1   if "saya" > "dia":
2       print("Ya") #Ya
3   else:
4       print("Tidak")
5
6   if "dua" == "dua":
7       print("Sama")   #Sama
```

Fungsi Len

Untuk mengetahui jumlah karakter dalam sebuah string, gunakan fungsi len(<string>). Untuk mengambil huruf terakhir, akses indeks ke len(<string>) - 1 karena indeks dimulai dari 0.

```
1   kalimat = "universitas kristen duta wacana yogyakarta"
2   print(len(kalimat)) #output 42
3
4   terakhir = kalimat[len(kalimat)-1]
5   print(terakhir) #output 'a'
6
7   #bisa juga menggunakan indeks -1
8   terakhir_versi2 = kalimat[-1]
9   print(terakhir_versi2)  #output 'a'
10  #atau menggunakan indeks -2 untuk huruf terakhir kedua
11  terakhir2 = kalimat[-2]
12  print(terakhir2)    #output 't'
```

Traversing String

Untuk dapat menampilkan string dengan cara ditampilkan huruf demi huruf adalah dengan menggunakan loop yang dilakukan per huruf dengan 2 cara:

- Dengan cara akses terhadap indeks

```
kalimat = "indonesia jaya"
i = 0
while i < len(kalimat):
    print(kalimat[i],end='')
    i += 1
```

- Dilakukan tanpa akses terhadap indeks otomatise

```
kalimat = "indonesia jaya"
for kal in kalimat:
    print(kal,end='')
```

String Slice

String slice adalah teknik untuk mengambil sebagian dari string (substring) dengan menentukan indeks mulai dan indeks akhir-1. Penulisannya menggunakan format <string>[awal:akhir], di mana bagian awal atau akhir bisa dikosongkan. Perhitungan indeks awal dimulai dari 0.

```
kalimat = "cerita rakyat"
awal = 0
akhir = 6
print(kalimat[awal:akhir])  #cerita
print(kalimat[7:len(kalimat)])  #rakyat
print(kalimat[:5])  #cerit
print(kalimat[5:])  #a rakyat
print(kalimat[:])   #cerita rakyat
```

String merupakan data yang bersifat immutable! Immutable adalah bahwa data tersebut tidak bisa diubah saat program berjalan, hanya bisa diinisialisasi saja

Di bawah ini merupakan table method String yang biasa digunakan, selain yang ada di table bawah ini dapat diakses di https://docs.python.org/library/stdtypes.html#string-methods :

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang dinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang dinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Dalam Python, operator + dapat digunakan untuk menggabungkan dua string, sedangkan operator * bisa digunakan untuk mengulang string sesuai jumlah perkaliannya. Contohnya seperti dibawah ini:

```
kata1 = "saya"
kata2 = "makan"
kata3 = kata1 + " " + kata2
print(kata3)     #hasil adalah penggabungan: saya makan
kata4 = "ulang"
print(kata4 * 4)     #hasil adalah ulangulangulangulang
kata4 = "ulang "
print(kata4 * 2)     #hasil adalah ulang ulang
```

MATERI 4 Parsing String

Parsing string adalah proses menelusuri dan memanipulasi bagian-bagian string untuk menemukan atau mengubah data tertentu. Misalnya, dari kalimat "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka", kita bisa mengambil tanggal dengan membagi string berdasarkan spasi, mencari bagian yang berisi angka, memecahnya dengan tanda '-' lalu menyusun ulang menjadi format 08/17/1945.

```
kalimat = "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka"

hasil = kalimat.split(" ")

for kal in hasil:
    if kal[0].isdigit():
        hasil2 = kal.split("-")
        print(hasil2[1]+"/"+hasil2[0]+"/"+hasil2[2])
```

MATERI 5 Pengantar RegEx

Dalam bab String, kita telah belajar berbagai teknik pengolahan string, termasuk dari file, namun teknik standar sering terasa rumit. Untuk itu, regular expression (regex) digunakan untuk mempermudah pencarian, penggantian, atau penghapusan string berdasarkan pola tertentu. Regex sangat kuat dalam mencari dan mengambil pola, meskipun sintaksnya cukup kompleks. Tidak semua bahasa pemrograman mendukung regex, tetapi Python mendukungnya melalui library re, dengan fungsi seperti search(). Sebagai contoh, kita bisa mencari semua string yang mengandung pola "From: " di file mbox-short.txt.

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
line=line.rstrip()
if re.search('From:', line):
count += 1
print(line)
print("Count: ",count)
```

Dari contoh diatas, sebenarnya re.search bisa digantikan dengan find() biasa, karena pola yang digunakan belum memanfaatkan kekuatan penuh regex. Namun jika dingin mencari baris diawali dengan pola “from”, maka kita harus mengubah parameter fungsi search pada re.search menjadi re.search(“from”).

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
line=line.rstrip()
if re.search('^From:', line):
count += 1
print(line)
print("Count: ",count)
```

MATERI 6 Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Libary Python

Sebelum menggunakan fungsi regex perlu diketahui terlebih dahulu meta character/specia; character dan kegunaanya pada pola regex seperti pada table dibawah ini:

Tabel 8.2: Escaped Character pada Regex

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Tabel 8.1: Special Character pada Python

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{}	Karakter dengan arti khusus dan escaped character	\{}d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
$	Dakhiri dengan	this$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{}d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{}d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{}d+	Minimal 1 s/d tak terhingga karakter
{}	Tepat sebanyak yang ada para {}	\{}d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
|	atau	\{}d|\{}s	1 digit atau 1 spasi

Dalam Python, terdapat beberapa karakter spesial (escaped characters) yang dijelaskan pada tabel 8.2. Selain itu, Python juga memungkinkan penggunaan himpunan karakter dengan simbol [], seperti yang ditunjukkan pada tabel 8.3.

Tabel 8.3: Himpunan Karakter pada Regex

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Pada Python terdapat 4 buah fungsi yang bisa dipakai untuk menggunakan Regex seperti pada tabel 8.4

Tabel 8.4: Fungsi Regex pada Python

Nama Fungsi	**Kegunaan**
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

BAGIAN 2: LATIHAN MANDIRI (60%)

https://github.com/Moses808/71251198_Moses_Merohito

SOAL 1

jupyter 71251198_Moses Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab

[2]:
```python
def is_anagram(kata1, kata2):
    kata1 = kata1.lower()
    kata2 = kata2.lower()

    return sorted(kata1) == sorted(kata2)

kata1 = input("Masukkan kata pertama: ")
kata2 = input("Masukkan kata kedua: ")

if is_anagram(kata1, kata2):
    print("Kedua kata adalah anagram.")
else:
    print("Kedua kata bukan anagram.")
```

```
Masukkan kata pertama:  kasur
Masukkan kata kedua:  rusak
Kedua kata adalah anagram.
```

Penjelasan

- Tujuan program:
  Program ini digunakan untuk mengecek apakah dua kata termasuk anagram atau bukan.
- Pengertian anagram:
  Anagram adalah dua kata yang memiliki huruf-huruf yang sama, tetapi urutannya berbeda.
- Fungsi is_anagram(kata1, kata2):
  Fungsi ini bertugas membandingkan dua kata untuk menentukan apakah keduanya anagram.

- Mengubah huruf menjadi kecil:
  Kedua kata diubah menjadi huruf kecil agar program tidak membedakan huruf besar dan huruf kecil.
- Mengurutkan huruf:
  Huruf-huruf pada kata pertama dan kata kedua diurutkan.
- Membandingkan hasil urutan:
  Jika hasil urutan huruf dari kedua kata sama, maka kedua kata adalah anagram.
  Jika berbeda, maka kedua kata bukan anagram.
- Input pengguna:
  Program meminta pengguna memasukkan kata pertama dan kata kedua.
- Proses pengecekan:
  Program memanggil fungsi is_anagram untuk memeriksa kedua kata tersebut.
- Output program:
  Jika anagram, program menampilkan:
  Kedua kata adalah anagram.
  Jika bukan, program menampilkan:
  Kedua kata bukan anagram.
- Contoh:
  Kata mata dan atma adalah anagram karena huruf-hurufnya sama.
  Kata mata dan tamu bukan anagram karena hurufnya berbeda.

SOAL 2

jupyter 71251198_Moses Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab

```
else:
    print("Kedua kata bukan anagram.")
```

```
Masukkan kata pertama:  kasur
Masukkan kata kedua:  rusak
Kedua kata adalah anagram.
```

[3]:
```
import re

kalimat = "Saya mau makan. Makan itu wajib. Mau siang atau m
kata_dicari = "makan"

kalimat = kalimat.lower()
kata_dicari = kata_dicari.lower()

daftar_kata = re.findall(r'\b\w+\b', kalimat)

jumlah = daftar_kata.count(kata_dicari)

print(kata_dicari, "ada", jumlah, "buah")
```

```
makan ada 3 buah
```

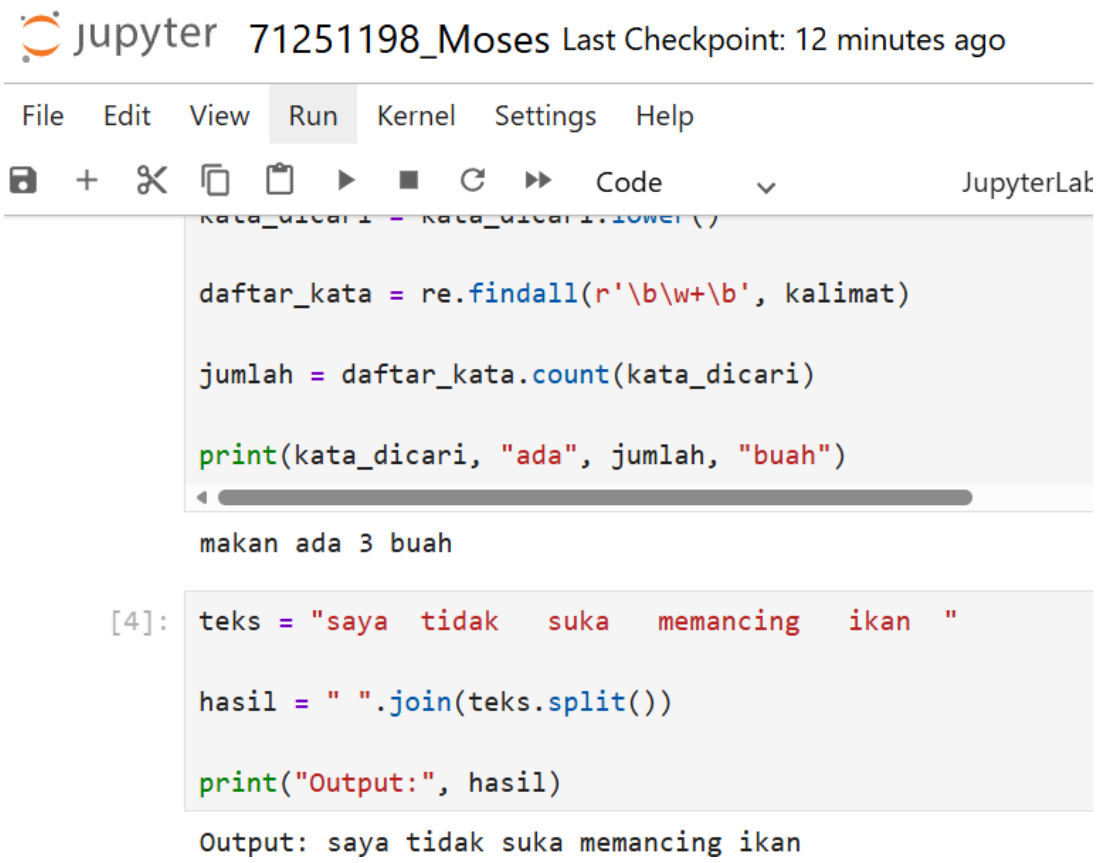
[ ]:

Penjelasan :

- Tujuan program:
  Program ini digunakan untuk menghitung berapa kali suatu kata muncul dalam sebuah kalimat atau string.
- Import re:
  Bagian ini digunakan untuk memanggil library re yang berfungsi membantu memproses teks.
- Variabel kalimat:
  Variabel ini berisi kalimat yang akan diperiksa.
- Variabel kata_dicari:
  Variabel ini berisi kata yang ingin dihitung jumlah kemunculannya.

- Mengubah huruf menjadi kecil:
  Kalimat dan kata yang dicari diubah menjadi huruf kecil agar huruf besar dan kecil dianggap sama.
- Mengambil semua kata:
  Program memisahkan kalimat menjadi kumpulan kata tanpa tanda baca.
- Menghitung jumlah kemunculan:
  Program menghitung berapa kali kata yang dicari muncul dalam kumpulan kata tersebut.
- Menampilkan hasil:
  Program menampilkan jumlah kemunculan kata yang dicari.
- Contoh hasil:
  Pada kalimat
  Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan
  kata makan muncul sebanyak 3 kali.
- Kesimpulan:
  Program bekerja dengan cara membaca kalimat, memecahnya menjadi kata-kata, lalu menghitung berapa kali kata tertentu muncul.

SOAL 3



Penjelasan :

- Tujuan program:
  Program ini digunakan untuk menghapus semua spasi yang berlebih pada sebuah string sehingga menjadi satu spasi normal.
- Variabel teks:
  Variabel ini berisi kalimat yang ingin diperbaiki spasinya.
- Fungsi split():
  Fungsi ini memisahkan kalimat menjadi kata-kata dan secara otomatis menghilangkan spasi yang berlebih.
- Fungsi join():
  Fungsi ini menggabungkan kembali kata-kata yang sudah dipisahkan dengan satu spasi saja.
- Variabel hasil:
  Variabel ini menyimpan kalimat yang sudah rapi tanpa spasi berlebih.
- Perintah print():
  Perintah ini digunakan untuk menampilkan hasil akhir ke layar.
- Hasil program:
  Kalimat yang awalnya memiliki banyak spasi akan berubah menjadi kalimat dengan satu spasi normal antar kata.
- Kesimpulan:
  Program bekerja dengan cara memisahkan kata-kata dalam string, lalu menggabungkannya kembali dengan satu spasi agar hasilnya lebih rapi.

SOAL 4

jupyter 71251198_Moses Last Checkpoint: 16 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab

makan ada 3 buah

[4]:
```
teks = "saya  tidak   suka   memancing   ikan  "

hasil = " ".join(teks.split())

print("Output:", hasil)
```

```
Output: saya tidak suka memancing ikan
```

[5]:
```
kalimat = input("Masukkan kalimat: ")

kata = kalimat.split()

terpendek = min(kata, key=len)
terpanjang = max(kata, key=len)

print("terpendek:", terpendek)
print("terpanjang:", terpanjang)
```

```
Masukkan kalimat:  ed snakes and a black frog in the pool
terpendek: a
terpanjang: snakes
```

Penjelasan :

- Tujuan program:
  Program ini digunakan untuk mencari kata terpendek dan kata terpanjang dari sebuah kalimat yang diinputkan.
- Variabel kalimat:
  Variabel ini digunakan untuk menyimpan kalimat yang dimasukkan oleh pengguna.
- Fungsi split():
  Fungsi ini memisahkan kalimat menjadi beberapa kata berdasarkan spasi.
  Contohnya, kalimat
  red snakes and a black frog in the pool
  akan diubah menjadi daftar kata.
- Variabel kata:
  Variabel ini menyimpan hasil pemisahan kalimat menjadi kumpulan kata.
- Fungsi min(kata, key=len):
  Fungsi ini digunakan untuk mencari kata yang jumlah hurufnya paling sedikit.
  Hasilnya disimpan dalam variabel terpendek.

- Fungsi max(kata, key=len):
  Fungsi ini digunakan untuk mencari kata yang jumlah hurufnya paling banyak.
  Hasilnya disimpan dalam variabel terpanjang.
- Variabel terpendek:
  Variabel ini menyimpan kata dengan panjang paling sedikit.
- Variabel terpanjang:
  Variabel ini menyimpan kata dengan panjang paling banyak.
- Perintah print():
  Perintah ini digunakan untuk menampilkan kata terpendek dan kata terpanjang ke layar.
- Contoh hasil:
  Jika inputnya adalah
  red snakes and a black frog in the pool
  maka hasilnya:
  terpendek: a
  terpanjang: snakes
- Kesimpulan:
  Program bekerja dengan cara memisahkan kalimat menjadi kata-kata, lalu membandingkan panjang setiap kata untuk menentukan mana yang paling pendek dan mana yang paling panjang.

SOAL 5

jupyter 71251198_Moses Last Checkpoint: 22 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python [conda e

```
Masukkan kalimat:  ed snakes and a black frog in the pool
terpendek: a
terpanjang: snakes
```

[6]:

```python
import re
from datetime import datetime

teks = """Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan
nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 1783-06-08) dan Ki
Hajar Dewantara (1889-05-02)."""

tanggal_ditemukan = re.findall(r'\d{4}-\d{2}-\d{2}', teks)

sekarang = datetime.now()

for tgl in tanggal_ditemukan:
    tanggal_obj = datetime.strptime(tgl, "%Y-%m-%d")
    selisih = (sekarang - tanggal_obj).days
    format_baru = tanggal_obj.strftime("%d-%m-%Y")

    print(f"Tanggal asli   : {tgl}")
    print(f"Format baru    : {format_baru}")
    print(f"Selisih hari   : {selisih} hari")
    print()
```

```
Tanggal asli   : 1945-08-17
Format baru    : 17-08-1945
Selisih hari   : 29465 hari

Tanggal asli   : 1785-11-11
Format baru    : 11-11-1785
Selisih hari   : 87817 hari

Tanggal asli   : 1783-06-08
Format baru    : 08-06-1783
Selisih hari   : 88704 hari

Tanggal asli   : 1889-05-02
Format baru    : 02-05-1889
Selisih hari   : 50025 hari
```

Penjelasan :

- Tujuan program:
  Program ini digunakan untuk mencari semua tanggal dalam sebuah teks yang memiliki format YYYY-MM-DD, lalu menampilkan kembali tanggal tersebut dalam format DD-MM-YYYY serta menghitung selisih harinya dengan tanggal sekarang.
- Import re:
  Bagian ini digunakan untuk memanggil library re yang berfungsi untuk mencari pola tertentu di dalam teks.
- Import datetime:
  Bagian ini digunakan untuk memanggil library datetime yang berfungsi untuk mengolah data tanggal dan waktu.
- Variabel teks:
  Variabel ini berisi kalimat atau paragraf yang di dalamnya terdapat beberapa tanggal.
- Fungsi re.findall():
  Fungsi ini digunakan untuk mencari semua teks yang sesuai dengan pola tanggal YYYY-MM-DD.
- Pola r'\d{4}-\d{2}-\d{2}':
  Pola ini berarti:
  empat digit angka untuk tahun,

dua digit angka untuk bulan,
dua digit angka untuk tanggal.
Contohnya seperti 1945-08-17.

- Variabel tanggal_ditemukan:
  Variabel ini menyimpan semua tanggal yang berhasil ditemukan dari teks.
- Variabel sekarang:
  Variabel ini berisi tanggal dan waktu saat program dijalankan.
- Perulangan for:
  Perulangan ini digunakan untuk memproses setiap tanggal yang ditemukan satu per satu.
- Fungsi datetime.strptime():
  Fungsi ini digunakan untuk mengubah tanggal yang masih berbentuk teks menjadi bentuk tanggal yang bisa dihitung.
- Menghitung selisih hari:
  Program mengurangkan tanggal sekarang dengan tanggal yang ditemukan, lalu mengambil jumlah harinya saja.
- Fungsi strftime("%d-%m-%Y"):
  Fungsi ini digunakan untuk mengubah format tanggal dari YYYY-MM-DD menjadi DD-MM-YYYY.
- Perintah print():
  Perintah ini digunakan untuk menampilkan tanggal asli, format baru, dan selisih harinya ke layar.
- Contoh hasil:
  Jika ditemukan tanggal 1945-08-17, maka format barunya menjadi 17-08-1945, lalu program menghitung berapa hari selisihnya dengan tanggal hari ini.
- Kesimpulan:
  Program bekerja dengan cara mencari semua tanggal dalam teks, mengubah format tanggal tersebut, lalu menghitung selisihnya dengan tanggal sekarang dalam satuan hari.

SOAL 6

jupyter 71251198_Moses Last Checkpoint: 26 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python [conda env:base] *

```
Format baru     : 02-05-1889
Selisih hari    : 50025 hari
```

[7]:

```python
import re
import random
import string

teks = """Berikut adalah daftar email dan nama pengguna dari mailing list:
anton@mail.com dimiliki oleh antonius
budi@gmail.co.id dimiliki oleh budi anwari
slamet@getnada.com dimiliki oleh slamet slumut
matahari@tokopedia.com dimiliki oleh toko matahari"""

daftar_email = re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', teks)

def generate_password(panjang=8):
    karakter = string.ascii_letters + string.digits
    return ''.join(random.choice(karakter) for _ in range(panjang))

for email in daftar_email:
    username = email.split('@')[0]
    password = generate_password()
    print(f"{email} username: {username} , password: {password}")
```

```
anton@mail.com username: anton , password: WjaJS9sR
budi@gmail.co.id username: budi , password: dWLUIfsm
slamet@getnada.com username: slamet , password: A1CHy1xE
matahari@tokopedia.com username: matahari , password: UMmNEHxe
```

Penjelasan :

- Tujuan program:
  Program ini digunakan untuk mencari semua teks yang berupa email di dalam sebuah string, lalu mengambil username dari setiap email dan membuat password acak sepanjang 8 karakter yang terdiri dari huruf dan angka.
- Import re:
  Bagian ini digunakan untuk memanggil library re.
  Library ini berfungsi untuk mencari pola teks tertentu, dalam hal ini pola email.

- Import random:
  Bagian ini digunakan untuk memanggil library random.
  Library ini berfungsi untuk memilih karakter secara acak saat membuat password.
- Import string:
  Bagian ini digunakan untuk memanggil library string.
  Library ini menyediakan kumpulan huruf besar, huruf kecil, dan angka yang akan dipakai dalam password.
- Variabel teks:
  Variabel ini berisi kalimat atau paragraf yang mengandung beberapa alamat email.
- Fungsi re.findall():
  Fungsi ini digunakan untuk mencari semua teks yang sesuai dengan pola email.
  Hasil pencarian akan disimpan dalam bentuk daftar email.
- Pola email:
  Pola ini dibuat agar program bisa mengenali susunan email yang benar, seperti:
  namauser@domain.com
  namauser@gmail.co.id
- Variabel daftar_email:
  Variabel ini menyimpan semua email yang berhasil ditemukan dari teks.
- Fungsi generate_password(panjang=8):
  Fungsi ini dibuat untuk menghasilkan password acak sepanjang 8 karakter.
- string.ascii_letters + string.digits:
  Bagian ini berisi semua huruf alfabet besar, huruf alfabet kecil, dan angka.
  Karakter-karakter inilah yang akan dipilih secara acak untuk membuat password.
- random.choice(karakter):
  Fungsi ini digunakan untuk mengambil satu karakter secara acak dari kumpulan huruf dan angka.
- Perulangan for:
  Perulangan ini digunakan untuk memproses setiap email yang ditemukan satu per satu.
- Mengambil username:
  Username diambil dari bagian sebelum tanda @ pada email.
  Contohnya:
  anton@mail.com memiliki username anton
  budi@gmail.co.id memiliki username budi
- Membuat password:
  Untuk setiap email, program akan membuat satu password acak yang panjangnya 8 karakter.
- Perintah print():
  Perintah ini digunakan untuk menampilkan email, username, dan password hasil generate.
- Contoh hasil:
  anton@mail.com username: anton , password: 8u78A2UD
  budi@gmail.co.id username: budi , password: bdP066Ld

- Catatan:
  Password yang dihasilkan akan selalu berbeda setiap kali program dijalankan karena dibuat secara acak.
- Kesimpulan:
  Program bekerja dengan cara mencari semua email dalam teks, mengambil username dari setiap email, lalu membuat password random 8 karakter menggunakan huruf dan angka.